

UNIVERSIDADE FEDERAL DE SANTA CATARINA

Departamento de Engenharia Elétrica e Eletrônica

EEL7522 - Processamento Digital de Sinais

Projeto 1: Processamento de Voz

Filtragem, Correlação Cruzada e Alteração de Taxa de Amostragem

Alunos:	Caio Missfeld Carlos (Matrícula: 22202674) Miguel Sória da Luz (Matrícula: 22100860)
Curso:	Engenharia Elétrica / Eletrônica
Repositório GitHub:	github.com/Caiobinha1/Projeto_PDS_1
Data:	2026

1. Introdução

Este relatório apresenta o desenvolvimento e as análises do Projeto 1 da disciplina de Processamento Digital de Sinais. O objetivo principal é aplicar conceitos fundamentais de filtragem linear, correlação de sinais e alteração da taxa de amostragem (upsampling e downsampling) em um sinal real de áudio de voz. As implementações foram projetadas em ambiente Scilab conforme solicitado, e validadas através de simulação em Python.

2. Sinal de Voz de Entrada (Original)

Antes de iniciarmos o processamento digital, analisamos o sinal de voz de entrada carregado. O gráfico abaixo apresenta o formato de onda do áudio original no domínio do tempo e o seu espectro de frequências obtido via FFT (Transformada Rápida de Fourier). No domínio do tempo, podemos visualizar a variação da amplitude do sinal e identificar as partes sonoras e os silêncios. No domínio da frequência, o espectro revela a distribuição de energia da voz, mostrando as frequências fundamentais (tons mais graves da fala) e os formantes (picos espectrais correspondentes à ressonância do trato vocal), que concentram a maior parte da energia da voz humana abaixo de 4 kHz.

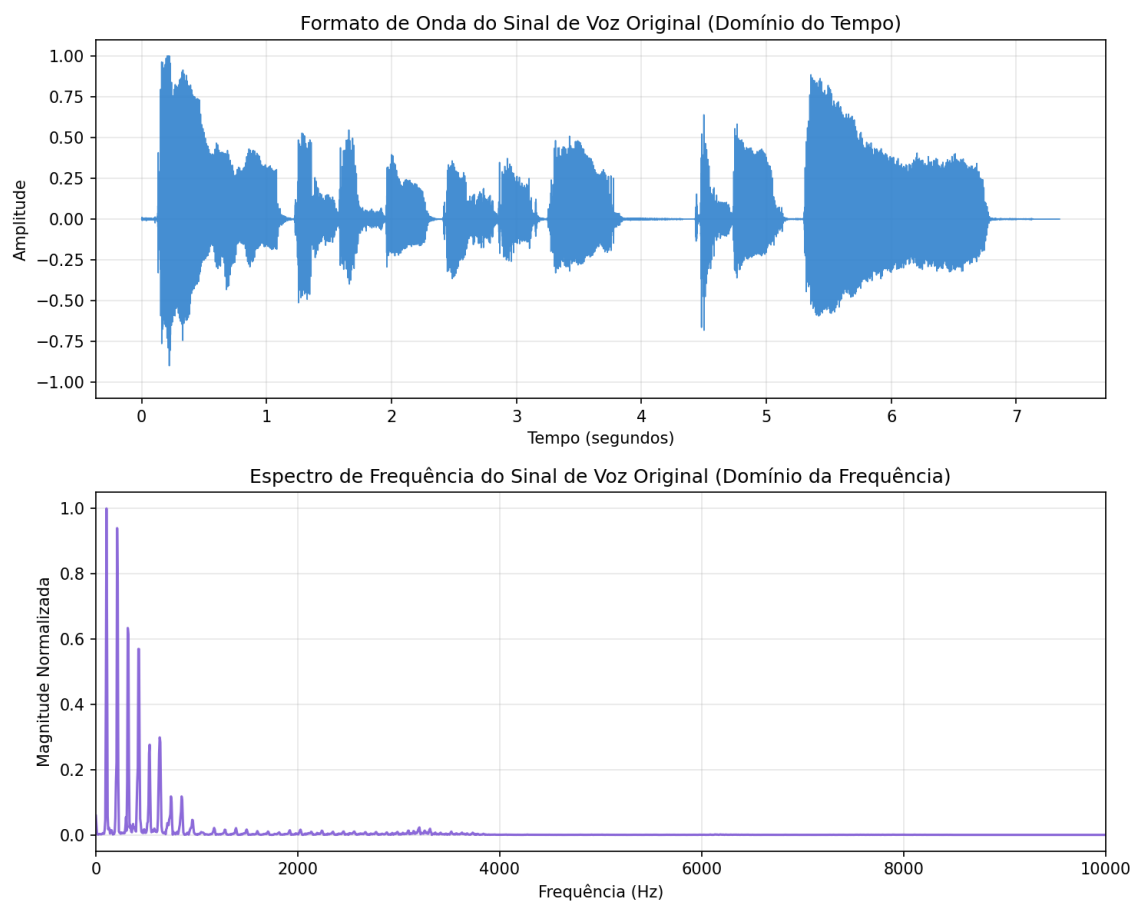


Figura 1: Sinal de voz original representado no domínio do tempo (onda) e da frequência (espectro).

3. Item 4.1: Filtragem com Esquecimento

O filtro de esquecimento é um filtro recursivo (IIR) de primeira ordem descrito pela equação de diferenças: $y[n] = x[n] + \alpha \cdot y[n-1]$. A função de transferência deste sistema é dada por $H(z) = 1 / (1 - \alpha \cdot z^{-1})$, o que indica a presença de um único polo localizado em $z = \alpha$ no plano complexo. A estabilidade do sistema depende do polo estar dentro do círculo unitário, ou seja, $|\alpha| < 1$. Testamos os valores de α propostos pelo professor:

Discussão dos Resultados:

- $\alpha = 0.98$: Coloca o polo muito perto do círculo unitário na frequência zero ($z = 1$). Funciona como um filtro passa-baixas extremamente forte. Ao ouvir o áudio, a voz fica muito abafada e grave, sendo difícil entender o que é dito, pois quase todos os agudos são eliminados.
- $\alpha = 0.5$: O polo se afasta do círculo unitário. Continua sendo um filtro passa-baixas, mas com uma frequência de corte bem maior. O áudio fica ligeiramente suavizado, mas a fala é compreendida perfeitamente.
- $\alpha = -0.98$: Coloca o polo perto do círculo unitário na frequência Nyquist ($z = -1$). Funciona como um passa-altas muito forte. O som perde todo o corpo e os graves, restando apenas um sussurro estridente e sibilante, com chiados metálicos dominando o sinal.
- $\alpha = -0.5$: Um filtro passa-altas mais suave. Os graves são levemente atenuados, dando destaque para os sons agudos e de respiração, mas mantendo a voz reconhecível.
- **Outros valores ($\alpha = 0.9$)**: Funciona como um passa-baixas intermediário, limpando chiados de alta frequência e mantendo a voz inteligível, embora levemente encorpada.

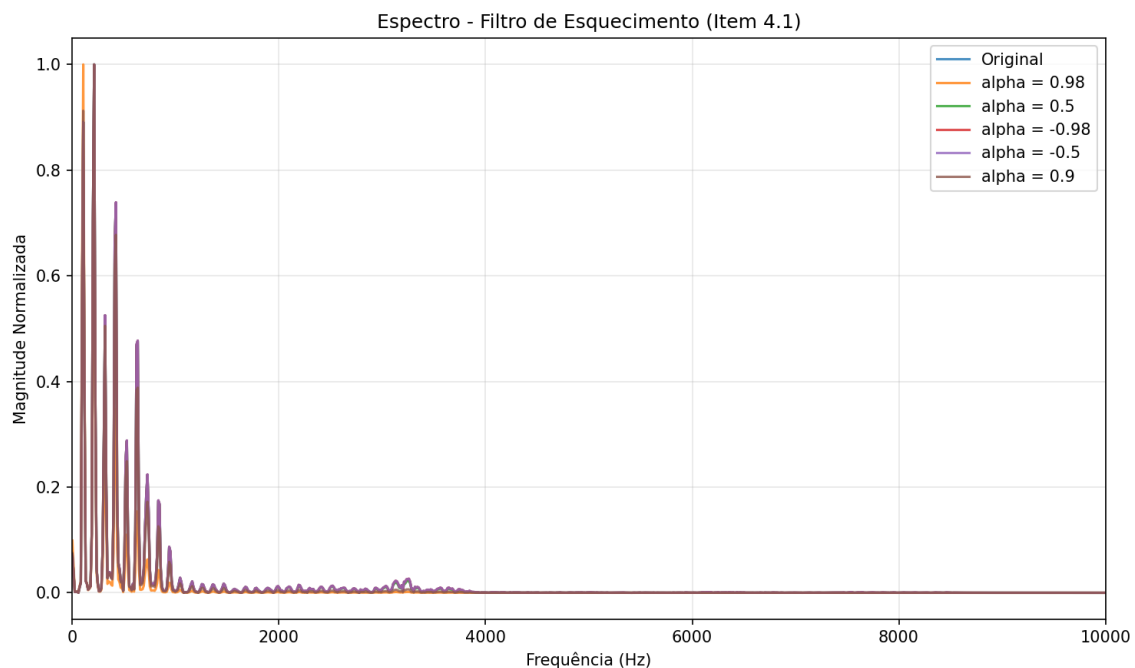


Figura 2: Espectro de frequência do sinal de voz sob efeito do Filtro de Esquecimento.

Código Fonte Scilab - Item 4.1:

```
// =====  
// PROJETO DE PDS - ITEM 4.1: FILTRO DE ESQUECIMENTO (FILTRO IIR DE 1ª ORDEM)  
// =====  
  
// Limpa o console e as variaveis do Scilab para começar do zero  
clc;  
clear;  
  
// Nome do arquivo de audio de entrada (localizado na pasta pai)  
arquivo_entrada = "../input_voice.wav";  
// Tenta carregar o audio de voz usando o wavread  
try  
    [x, Fs] = wavread(arquivo_entrada);  
catch  
    disp("Erro ao usar wavread. Tentando usar a funcao audioread...");  
    [x, Fs] = audioread(arquivo_entrada);  
end  
  
// Verifica o tamanho do sinal carregado para ver se e estereo ou mono  
[linhas, colunas] = size(x);  
// Vamos garantir que estamos trabalhando apenas com um canal (sinal mono)  
if linhas > 1 then  
    sinal_mono = x(1, :);  
else  
    sinal_mono = x'; // Se for vetor coluna, transformamos em linha  
end  
  
// Comprimento total do sinal de voz (numero de amostras)  
N = length(sinal_mono);  
// Lista com os valores de alpha que o professor pediu para testarmos (incluindo o 0.9)  
alphas = [0.98, 0.5, -0.98, -0.5, 0.9];  
// Loop para passar por cada valor de alpha da lista  
for i = 1:length(alphas)  
    alpha = alphas(i);  
    disp("Processando para alpha = " + string(alpha) + "...");  
    // Cria um vetor de zeros para guardar o sinal de saida  
    y = zeros(1, N);  
    // A primeira amostra do sinal filtrado recebe a primeira amostra do sinal de entrada  
    y(1) = sinal_mono(1);  
    // Aplica a equacao de diferencas:  $y[n] = x[n] + \alpha * y[n-1]$   
    for n = 2:N  
        y(n) = sinal_mono(n) + alpha * y(n-1);  
    end  
    // Normalizacao para garantir que o som nao estoure e fique entre -1 e 1  
    valor_minimo = min(y);  
    valor_maximo = max(y);  
    normalix = max(abs(valor_minimo), abs(valor_maximo));  
    y_normalizado = y / normalix;  
    // Monta o nome do arquivo de saida direcionando para a pasta de audios processados  
    if alpha < 0 then  
        nome_saida = "../Audios_Processados/output_4_1_alpha_menos_" + string(abs(alpha)) + ".wav";  
    else  
        nome_saida = "../Audios_Processados/output_4_1_alpha_" + string(alpha) + ".wav";  
    end  
    // Salva o audio processado na pasta correta  
    try  
        wavwrite(y_normalizado, Fs, nome_saida);  
    catch  
        audiowrite(nome_saida, y_normalizado', Fs);  
    end  
end
```

```
        disp("Arquivo salvo com sucesso: " + nome_saida);  
    end  
    disp("Fim do processamento do item 4.1!");
```

4. Item 4.2: Filtragem de Média Móvel

O filtro de média móvel é um filtro de resposta ao impulso finita (FIR) cuja equação de diferenças é dada por $y[n] = (1/M) * \sum x[n-k]$ para k variando de 0 até $M-1$. Ele calcula a média aritmética das últimas M amostras. Sua resposta em frequência se comporta de maneira similar a uma função sinc, agindo como um filtro passa-baixas clássico. À medida que aumentamos o tamanho da janela M , a largura de banda diminui drasticamente.

Discussão dos Resultados:

- **M = 10 (Valor Extra):** Apresenta um efeito de abafamento quase imperceptível. Por ser uma janela muito curta (apenas 10 amostras), a frequência de corte do filtro é extremamente alta. Ele atua apenas suavizando ruídos de altíssima frequência (como chiados metálicos muito agudos), mantendo a voz original praticamente intacta e perfeitamente inteligível.
- **M = 50:** O áudio sofre uma atenuação de agudos leve. Soa como se a pessoa estivesse falando um pouco distante do microfone. Os ruídos de chiado de alta frequência são removidos com eficácia.
- **M = 100:** O efeito de abafamento fica muito forte. A voz parece vir de trás de uma porta fechada. A perda de componentes de alta frequência prejudica bastante a distinção de consoantes como 's', 't' e 'p'.
- **M = 1000:** O sinal é severamente suavizado. Como 1000 amostras a 16 kHz representam um intervalo de tempo de 62.5 ms (ou 22.7 ms a 44.1 kHz), o filtro remove praticamente toda a informação de voz, restando apenas um murmúrio de baixíssima frequência (um zumbido grave). A voz torna-se totalmente incompreensível.

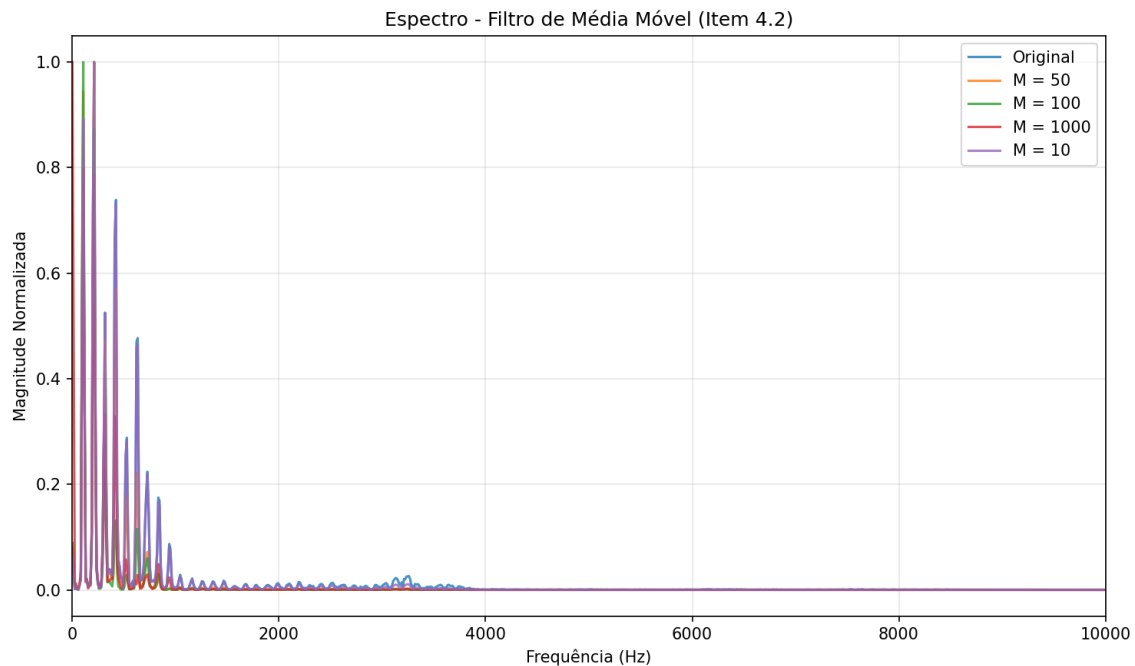


Figura 3: Espectro de frequência do sinal de voz sob efeito do Filtro de Média Móvel.

Código Fonte Scilab - Item 4.2:

```
// =====  
// PROJETO DE PDS - ITEM 4.2: FILTRO DE MÉDIA MÓVEL (FILTRO FIR)  
// =====  
  
// Limpa o console e as variaveis para começar limpo  
clc;  
clear;  
  
// Nome do arquivo de audio de entrada (localizado na pasta pai)  
arquivo_entrada = "../teste.wav";  
// Tenta carregar o audio usando o wavread  
try  
    [x, Fs] = wavread(arquivo_entrada);  
catch  
    disp("Erro ao usar wavread. Tentando usar a funcao audioread...");  
    [x, Fs] = audioread(arquivo_entrada);  
end  
  
// Pega apenas um canal de audio (mono)  
[linhas, colunas] = size(x);  
if linhas > 1 then  
    sinal_mono = x(1, :);  
else  
    sinal_mono = x'; // Garante que seja um vetor de linha  
end  
  
N = length(sinal_mono);  
// Valores de M (incluindo o valor extra 10 para refletir os outros valores de M)  
Ms = [50, 100, 1000, 10];  
// Loop para rodar o filtro para cada valor de M  
for i = 1:length(Ms)  
    M = Ms(i);  
    disp("Processando filtro de media movel para M = " + string(M) + "...");  
    // Filtro h e um vetor de 1s de tamanho M, dividido por M (calcula a media)  
    h = ones(1, M) / M;  
    // Faz a convolucao do sinal de voz com o nosso filtro h  
    y_completo = convol(h, sinal_mono);  
    // Cortamos para ficar com o mesmo tamanho original N de amostras  
    y = y_completo(1:N);  
    // Normalizacao antes de salvar (faixa de -1 a 1)  
    valor_minimo = min(y);  
    valor_maximo = max(y);  
    normalix = max(abs(valor_minimo), abs(valor_maximo));  
    y_normalizado = y / normalix;  
    // Nome do arquivo direcionando para a pasta de audios processados  
    nome_saida = "../Audios_Processados/output_4_2_M_" + string(M) + ".wav";  
    // Salva o audio processado  
    try  
        wavwrite(y_normalizado, Fs, nome_saida);  
    catch  
        audiowrite(nome_saida, y_normalizado', Fs);  
    end  
    disp("Arquivo salvo com sucesso: " + nome_saida);  
end  
  
disp("Fim do processamento do item 4.2!");
```

5. Item 4.3: Correlação de Sinais

A correlação cruzada é uma ferramenta matemática usada para medir a semelhança entre dois sinais em função do deslocamento temporal. No projeto, extraímos um segmento de 1 segundo (entre os segundos 2 e 3 do áudio original) e calculamos a correlação desse segmento com o sinal original inteiro. No Scilab, esse cálculo é otimizado através da convolução do sinal de voz com o segmento invertido no tempo.

Interpretação dos Resultados:

O gráfico exibe a amplitude de correlação. Podemos notar claramente um **pico máximo de energia** no exato instante em que o segmento se alinha com sua contraparte original dentro do áudio de voz. Esse pico ocorre próximo ao índice correspondente ao tempo de início do segmento original (segundo 2). Eventuais outros picos menores no gráfico indicam momentos em que o locutor pronunciou fonemas ou palavras com características espectrais semelhantes às contidas no segmento extraído. Este método forma a base para sistemas simples de detecção de padrões, busca em áudio e reconhecimento de voz.

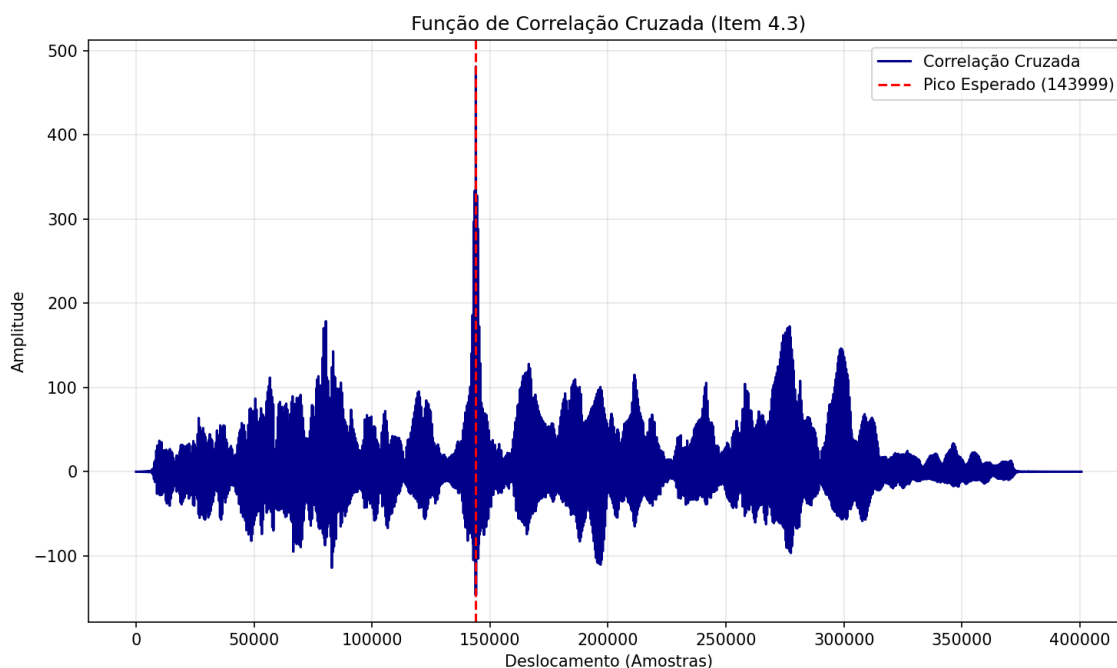


Figura 4: Função de Correlação Cruzada entre o segmento de 1s e o sinal de voz completo.

Código Fonte Scilab - Item 4.3:

```
// =====
// PROJETO DE PDS - ITEM 4.3: CORRELAÇÃO DE SINAIS
// =====

// Limpa tudo para começar limpo
clc;
clear;

// Arquivo de audio de entrada (localizado na pasta pai)
arquivo_entrada = "../teste.wav";
// Tenta carregar o audio usando o wavread ou audioread
try
    [x, Fs] = wavread(arquivo_entrada);
catch
    disp("Erro ao usar wavread. Tentando usar a funcao audioread...");
    [x, Fs] = audioread(arquivo_entrada);
end

// Garante que o sinal seja mono
[linhas, colunas] = size(x);
if linhas > 1 then
    sinal_mono = x(1, :);
else
    sinal_mono = x'; // Vetor de linha
end

N = length(sinal_mono);
// Tempo total do sinal em segundos
tempo_total = N / Fs;
disp("Tempo total do sinal: " + string(tempo_total) + " segundos.");
// Segmentar 1 segundo do sinal, entre os segundos 2 e 3 do audio original
n_inicio = round(2 * Fs) + 1;
n_fim = round(3 * Fs);
// Extraí o trecho do audio correspondente a esse intervalo de 1 segundo
segmento = sinal_mono(n_inicio:n_fim);
L_seg = length(segmento);
disp("Segmento extraído de " + string(n_inicio) + " ate " + string(n_fim) + " amostras.");
// Calcula a correlacao cruzada completa invertendo o segmento
segmento_invertido = segmento($:-1:1);
correlacao = convol(segmento_invertido, sinal_mono);
// Cria o vetor de deslocamento para o grafico
deslocamento_amstras = 1:length(correlacao);
// Plota o grafico da correlacao
clf(); // Limpa qualquer grafico aberto
plot(deslocamento_amstras, correlacao, "b");
xtitle("Funcao de Correlacao Cruzada", "Deslocamento (Amstras)", "Amplitude da Correlacao");
// Desenha a linha vertical no pico de correlacao maxima esperada (n_inicio + L_seg - 1)
indice_pico_esperado = n_inicio + L_seg - 1;
plot([indice_pico_esperado, indice_pico_esperado], [min(correlacao), max(correlacao)], "r--");
legend(["Correlacao Calculada", "Ponto de Alinhamento (Pico)"]);
disp("Fim do processamento do item 4.3. O grafico foi gerado!");
```

6. Item 4.4: Alteração de Taxa

Este item analisa as alterações nas características temporais e espectrais do sinal sob mudanças de taxa:

a) Dobro da taxa (Upsampling por fator 2):

Ao inserir uma amostra com valor zero entre duas amostras originais, o comprimento do sinal dobra. No domínio do tempo, o sinal fica mais espalhado. Ao salvar e ouvir este áudio na taxa F_s original, percebe-se a voz tocando na **metade da velocidade normal (câmera lenta)** e com um **tom de voz uma oitava mais grave (som grosso)**. Além disso, as amostras nulas causam descontinuidade rápida de sinal, introduzindo alta frequência artificial. No espectro, isso corresponde ao **espelhamento espectral** (imagens harmônicas adicionais). O espectro original é comprimido e sua réplica aparece nas frequências mais altas.

b) Redução da taxa (Downsampling por fator 2):

Eliminar as amostras de índice ímpar reduz o tamanho do sinal pela metade. No domínio do tempo, o sinal fica comprimido. Ao ouvir este arquivo na taxa F_s original, a voz toca no **dobro da velocidade normal** e com o **tom de voz uma oitava mais agudo (efeito Alvin e os Esquilos)**. Do ponto de vista espectral, a eliminação de amostras reduz a frequência de amostragem efetiva pela metade. Se o sinal possuir frequências acima da nova frequência Nyquist ($F_s/4$), ocorre o fenômeno de **aliasing (mascaramento espectral)**, onde frequências agudas são 'dobradas' sobre as baixas, causando distorções sonoras irreversíveis no áudio final.

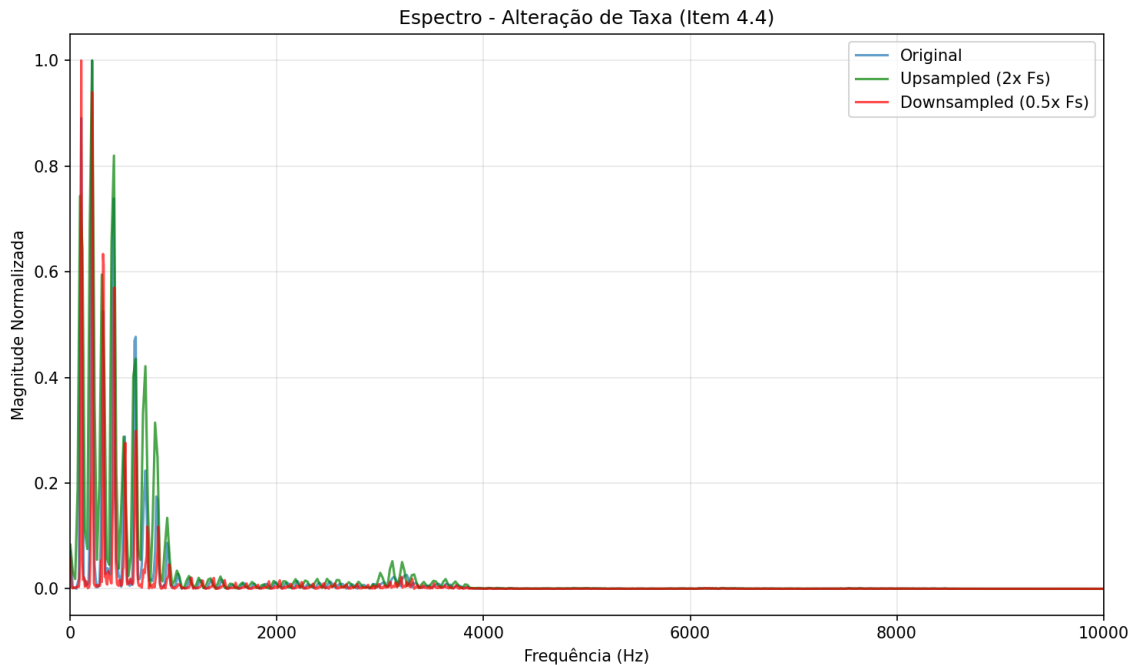


Figura 5: Espectro de frequência do sinal de voz sob efeito das Alterações de Taxa.

Código Fonte Scilab - Item 4.4:

```
// =====
// PROJETO DE PDS - ITEM 4.4: ALTERAÇÃO DE TAXA (REAMOSTRAGEM POR FATOR 2)
// =====

// Limpa tudo para começar limpo
clc;
clear;

// Arquivo de audio de entrada (localizado na pasta pai)
arquivo_entrada = "../teste.wav";
// Tenta carregar o audio usando o wavread ou audioread
try
    [x, Fs] = wavread(arquivo_entrada);
catch
    disp("Erro ao usar wavread. Tentando usar a funcao audioread...");
    [x, Fs] = audioread(arquivo_entrada);
end

// Garante que o sinal seja mono
[linhas, colunas] = size(x);
if linhas > 1 then
    sinal_mono = x(1, :);
else
    sinal_mono = x'; // Vetor de linha
end

N = length(sinal_mono);

// =====
// PARTE A: DOBRAR A TAXA (UPSAMPLING / EXPANSÃO POR FATOR 2)
// =====
disp("Processando Upsampling (insercao de zeros)...");
// Criamos um novo vetor de zeros com o dobro do tamanho do sinal original
y_up = zeros(1, 2 * N);
// Colocamos as amostras originais nas posicoes impares (1, 3, 5, ...)
y_up(1:2:$) = sinal_mono;
// Normalizacao antes de salvar (faixa de -1 a 1)
normalix_up = max(abs(min(y_up)), abs(max(y_up)));
y_up_normalizado = y_up / normalix_up;
// Salva o audio processado na pasta correta
nome_saida_up = "../Audios_Processados/output_4_4_upsampled.wav";
try
    wavwrite(y_up_normalizado, Fs, nome_saida_up);
catch
    audiowrite(nome_saida_up, y_up_normalizado', Fs);
end
disp("Arquivo de upsampling salvo: " + nome_saida_up);

// =====
// PARTE B: REDUZIR A TAXA (DOWNSAMPLING / DECIMAÇÃO POR FATOR 2)
// =====
disp("Processando Downsampling (remocao de amostras impares)...");
// Mantemos apenas as amostras de indices pares (2, 4, 6, ...), eliminando as impares.
y_down = sinal_mono(2:2:$);
// Normalizacao antes de salvar (faixa de -1 a 1)
normalix_down = max(abs(min(y_down)), abs(max(y_down)));
y_down_normalizado = y_down / normalix_down;
// Salva o audio processado na pasta correta
nome_saida_down = "../Audios_Processados/output_4_4_downsampled.wav";
try
    wavwrite(y_down_normalizado, Fs, nome_saida_down);
catch
```

```
    audiowrite(nome_saida_down, y_down_normalizado', Fs);  
end  
disp("Arquivo de downsampling salvo: " + nome_saida_down);  
disp("Fim do processamento do item 4.4!");
```